

Taller Práctico de Spring Boot: Sistema de Gestión de Proyectos (JPA, REST y Relaciones)

Este taller está diseñado para que implementes un backend utilizando **Spring Boot**, **Spring Data JPA**, **Spring Web** y una base de datos relacional (**PostgreSQL** o **MySQL**). El tema del taller es un **Sistema de Gestión de Proyectos**, donde administramos proyectos, empleados y las tareas asignadas a los mismos.

Objetivos del Taller

1. Configurar un proyecto Spring Boot desde cero utilizando **Spring Initializr**.
 2. Agregar y configurar las dependencias necesarias en Maven (`pom.xml`), incluyendo el driver de base de datos.
 3. Configurar la conexión a la base de datos en `application.properties`.
 4. Mapear entidades relacionales utilizando anotaciones de JPA (`@Entity`, `@Table`, `@Id`, `@ManyToOne`, `@ManyToMany`, etc.).
 5. Implementar la arquitectura en capas: Entidades -> Repositorios -> Servicios -> Controladores.
 6. Probar los servicios REST mediante peticiones HTTP.
 7. Subir el proyecto a un repositorio en **GitHub**.
-



Puntos de Control y Capturas (WhatsApp)

A lo largo de este taller, deberás enviar capturas de pantalla por WhatsApp en los momentos indicados con el ícono . Cada captura debe verse completa (incluyendo tu entorno de desarrollo o base de datos).

Paso 1: Creación del Proyecto y Configuración Básica

1.1 Descarga desde Spring Initializr

1. Ingresa a [Spring Initializr](#).
2. Configura las siguientes opciones:
 - **Project:** Maven
 - **Language:** Java
 - **Spring Boot:** Versión estable recomendada (ej. 3.x.x o 4.x.x)
 - **Project Metadata:**
 - **Group:** `com.krakedev`

- **Artifact:** `proyectos`
 - **Name:** `proyectos`
 - **Package name:** `com.krakedev.proyectos`
 - **Packaging:** Jar
 - **Java:** Versión compatible con tu JDK (ej. 17, 21 o 25).
3. En la sección **Dependencies** (a la derecha), busca y agrega:
 - **Spring Web** (para crear endpoints REST)
 - **Spring Data JPA** (para interactuar con la base de datos)
 4. Haz clic en **Generate** para descargar el archivo `.zip`. Descomprímelo y ábrelo en tu IDE (Eclipse, Spring Tool Suite, IntelliJ o VS Code).
-

1.2 Agregar el Driver de Base de Datos en Maven

Paso 2: Creación y Configuración de la Base de Datos

1. Abre tu gestor de base de datos (pgAdmin o MySQL Workbench).
 2. Crea una nueva base de datos llamada `proyectos_db`.
 - En PostgreSQL: `CREATE DATABASE proyectos_db;`
 - En MySQL: `CREATE DATABASE proyectos_db;`
-

Paso 3: Configuración del Archivo

`application.properties`

Ubica el archivo en `src/main/resources/application.properties` y configúralo con los datos de acceso a tu base de datos:

Paso 4: Estructura de Paquetes y Entidades JPA

Para mantener un código limpio y profesional, utilizaremos la siguiente estructura de paquetes dentro de `src/main/java/com/krakedev/proyectos`:

- `com.krakedev.proyectos.entidades` (Clases JPA que representan tablas)
 - `com.krakedev.proyectos.repositories` (Interfaces que extienden JpaRepository)
 - `com.krakedev.proyectos.services` (Clases de lógica de negocio)
 - `com.krakedev.proyectos.controllers` (Clases controladoras REST)
-

4.1 Entidades a Crear

El modelo relacional consta de tres entidades:

1. **Proyecto:** Un proyecto en desarrollo.
2. **Empleado:** Un colaborador de la empresa.

3. **Tarea:** Un requerimiento específico asignado a un proyecto, en el cual pueden colaborar uno o más empleados (Relación intermedia).

1. Clase `Proyecto.java`

Créala en el paquete `com.krakedev.proyectos.entidades`.

- **Mapeo:** `@Entity, @Table(name = "proyectos")`
- **Atributos:**
 - `id (int): @Id, @GeneratedValue(strategy = GenerationType.IDENTITY)`
 - `nombre (String): @Column(nullable = false, length = 100)`
 - `descripcion (String): @Column(length = 250)`
 - `fechaInicio (LocalDate): @Column(nullable = false)`
 - `tareas (List): @OneToMany(mappedBy = "proyecto")` y la anotación `@JsonIgnore` (para evitar bucles infinitos al serializar a JSON).
- Incluye el constructor vacío, constructor con parámetros (sin incluir colecciones), getters y setters.

2. Clase `Empleado.java`

Créala en el paquete `com.krakedev.proyectos.entidades`.

- **Mapeo:** `@Entity, @Table(name = "empleados")`
- **Atributos:**
 - `id (int): @Id, @GeneratedValue(strategy = GenerationType.IDENTITY)`
 - `nombre (String): @Column(nullable = false, length = 100)`
 - `cargo (String): @Column(nullable = false, length = 50)`
 - `tareas (List): @ManyToMany(mappedBy = "empleados")` y `@JsonIgnore`.
- Incluye constructor vacío, constructor con parámetros, getters y setters.

3. Clase `Tarea.java` (Entidad Relacional)

Créala en el paquete `com.krakedev.proyectos.entidades`. Esta entidad gestiona las claves foráneas de la relación:

- **Mapeo:** `@Entity, @Table(name = "tareas")`
- **Atributos:**
 - `id (int): @Id, @GeneratedValue(strategy = GenerationType.IDENTITY)`
 - `descripcion (String): @Column(nullable = false, length = 200)`
 - `fechaLimite (LocalDate): @Column(nullable = false)`
 - `costoEstimado (double): @Column(nullable = false)`
 - `proyecto (Proyecto): @ManyToOne con @JoinColumn(name = "proyecto_id", nullable = false).`
 - `empleados (List): Relación @ManyToMany con @JoinTable para definir la tabla intermedia:`
 - `java`
 - `@ManyToMany`
 - `@JoinTable (`
 - `name = "tarea_empleados",`
 - `joinColumns = @JoinColumn(name = "tarea_id"),`
 - `inverseJoinColumns = @JoinColumn(name = "empleado_id")`

-)
- `private List<Empleado> empleados;`
- Incluye constructor vacío, constructor con parámetros, getters y setters.

IMPORTANT

 **CAPTURA (WhatsApp):** Muestra el código fuente de tu entidad relacional `Tarea.java` mostrando claramente las relaciones `@ManyToOne` y `@ManyToMany` con `@JoinTable`.

Paso 5: Primera Ejecución y Creación de Tablas

1. Ejecuta tu proyecto Spring Boot (como Java Application desde la clase principal `ProyectosApplication.java`).
2. Verifica la consola del IDE. Al iniciar la aplicación, Hibernate debe leer las entidades y crear automáticamente las tablas en la base de datos basándose en el mapeo JPA.
3. Ve a tu gestor de base de datos y refresca la base de datos `proyectos_db`.

IMPORTANT

 **CAPTURA (WhatsApp):** Muestra la consola de tu IDE donde se observen las sentencias SQL de Hibernate creando las tablas (`CREATE TABLE...`).

Paso 6: Implementación de la Capa de Acceso a Datos (Repositories)

Crea las interfaces de repositorio en el paquete `com.krakedev.proyectos.repositories`:

Paso 7: Implementación de la Capa de Servicios (Services)

Crea las clases de servicio en `com.krakedev.proyectos.services` para manejar la lógica de negocio (CRUD):

Paso 8: Implementación de la Capa de Controladores REST (Controllers)

Crea las clases controladoras en el paquete `com.krakedev.proyectos.controllers` para exponer los endpoints de la API:

1. **`ProyectoController.java`** (Exponer en `/api/proyectos`)
 - Métodos POST (`/`), GET (`/`), GET (`//{id}`), PUT (`//{id}`), DELETE (`//{id}`).
2. **`EmpleadoController.java`** (Exponer en `/api/empleados`)
 - Métodos POST (`/`), GET (`/`), GET (`//{id}`), PUT (`//{id}`), DELETE (`//{id}`).
3. **`TareaController.java`** (Exponer en `/api/tareas`)

- Métodos POST (/) para guardar tareas (asociándoles proyectos y empleados existentes) y GET (/) para listarlas.

Asegúrate de manejar respuestas adecuadas utilizando `ResponseEntity<?>` y capturar excepciones con bloques `try-catch`, retornando un estado `500 Internal Server Error` con el detalle del error en caso de fallo.

Paso 9: Pruebas del Backend (Postman / Insomnia)

Utiliza un cliente REST (Postman o Insomnia) para realizar el flujo de datos completo. Realiza las siguientes solicitudes en orden:

1. Registrar un Empleado (POST)

- **URL:** `http://localhost:8080/api/empleados`
- **Body (JSON):**
- json
- {
- `"nombre": "Diana Rojas",`
- `"cargo": "Desarrollador Senior"`
- }

2. Registrar un Proyecto (POST)

- **URL:** `http://localhost:8080/api/proyectos`
- **Body (JSON):**
- json
- {
- `"nombre": "E-Commerce Plataforma",`
- `"descripcion": "Desarrollo del nuevo portal de ventas online",`
- `"fechaInicio": "2026-05-20"`
- }

3. Registrar una Tarea con relaciones (POST)

Para crear una tarea y asociarla al proyecto creado y al empleado creado, debes pasar sus identificadores (IDs) en el JSON. Asumiendo que el Proyecto tiene ID `1` y el Empleado tiene ID `1`:

- **URL:** `http://localhost:8080/api/tareas`
- **Body (JSON):**
- json
- {
- `"descripcion": "Diseñar e implementar el modelo de datos para la base de datos",`
- `"fechaLimite": "2026-06-15",`
- `"costoEstimado": 450.50,`
- `"proyecto": {`
- `"id": 1`

- },
- "empleados": [
- {
- "id": 1
- }
-]
- }

4. Consultar todas las Tareas (GET)

- **URL:** `http://localhost:8080/api/tareas`
- Verifica que el JSON devuelto contenga los datos de la tarea, la información del proyecto asociado y la lista de empleados asignados.

IMPORTANT

 **CAPTURA (WhatsApp):** Muestra el resultado exitoso en Postman/Insomnia de la consulta GET a `/api/tareas` mostrando el JSON completo con las relaciones.

Paso 10: Repositorio en GitHub y Entrega Final



Entrega Final

- Envía el **enlace de tu repositorio de GitHub**
- **Capturas de pantalla** solicitadas a través de WhatsApp.